Network and Digital Forensics

Investigating a Web Server Compromise

Case study: CyberDefenders Web Investigation Lab

Team members:
Shreeya Madulkar - 16010124163
Sweekriti Maheshwari - 16010124164

Branch: Computer Engineering

Cyber Forensics & Laws

K. J. Somaiya School of Engineering

Repository: https://github.com/ShreeyaMadulkar/web-investigation-dfir

Index

Sr No.	Contents	Page No.
1	Introduction	3
2	Objectives and Scope	3
3	Tools Used	4
4	Methodology	4
5	Findings	5
6	Timeline	18
7	Indicators of Compromise	19
8	MITRE ATT&CK Mapping	20
9	Impact Assessment	20
10	Recommendations	21
11	Limitations	21
12	Conclusion	21
13	References	21
A	Appendix A: Lab Questions and Answers	22

1. Introduction

This project investigates a simulated attack on BookWorld, an online bookstore. We were given a packet capture (PCAP) of the network traffic and used Wireshark and related tools to find out what happened.

The work follows the CyberDefenders Web Investigation lab, which is a training exercise. The incident, the company and the data are simulated. The goal is to show a real forensic method: start with the network evidence, find the attacker, trace each step of the attack and report only what the evidence proves.

Scenario: BookWorld is a large online bookstore. Late one evening an automated alert reported a sudden rise in database queries and server load. As the analyst on duty in the Security Operations Center (SOC), the job is to find how the attack happened, judge how much customer data was exposed and decide whether the attacker gained further access to BookWorld systems.

2. Objectives

Aim: To analyse a captured network traffic file (PCAP) from a simulated attack on an online bookstore and to rebuild how the attack happened using standard network forensics tools and methods.

- **Identify the attacker and the target.** Find the attacker IP address, the attacked server and the attacker's apparent location.
- **Find the attack vector.** Identify the vulnerable page and the first SQL injection attempt.
- **Identify the tools used.** Recognise the scanner and the SQL injection tool from the traffic.
- **Assess the data exposed.** Find which database and table the attacker read.
- **Check for admin access.** Find the hidden directory, the login attempts and whether a login worked.
- **Check for further control.** Find any uploaded malicious file, extract it, hash it and check whether it was run.
- **Document the incident.** Build a timeline, a list of indicators of compromise (IOCs) and a MITRE ATT&CK mapping.
- **Recommend fixes.** Suggest how BookWorld can stop a similar attack.

3. Tools Used

Tool	How we used it
Wireshark	Statistics (Conversations, Endpoints, Protocol Hierarchy, HTTP Requests, I/O Graph), display filters and Follow HTTP Stream.
tshark	Command line export of HTTP requests and grouping requests by User-Agent.
NetworkMiner	Searching the PCAP for the uploaded file and extracting it, with its frame number, size and source.
Python 3	Our script extract_iocs.py pulls IP addresses, URLs, domains and hashes from tshark output into a CSV.
VirusTotal	Hash lookup for the uploaded file (hash only, the file was not uploaded).
PowerShell	Get-FileHash for the SHA256 of the extracted file and Import-Csv to check the output of our script.
IPinfo	IP lookup for the approximate location of the attacker IP.

4. Methodology

- Open the PCAP and review the overview statistics (Conversations, Endpoints, Protocol Hierarchy).
- Find the busiest hosts and separate normal traffic from suspicious traffic.
- Review the HTTP request list and the User-Agent values to identify tools.
- Filter and follow streams to study the SQL injection requests and responses.
- Review POST requests to study the admin logins and the file upload.
- Check what happened after the upload.
- Extract the uploaded file with NetworkMiner, hash it with PowerShell and check the hash on VirusTotal.
- Build the timeline, the IOC table and the MITRE ATT&CK mapping.
- Run the Python script to extract IOCs from the exported requests.

5. Findings

5.1 Key results

Item	Result
Attacker IP	111.224.250.131
Victim server	73.124.22.98 (bookworldstore.com)
Scanning tool	Gobuster 3.6 (41,527 requests)
SQL injection tool	sqlmap 1.8.3 against /search.php
Data exposed	bookworld_db.customers table
Admin access	Yes, after three failed attempts
Malicious file	NVri2vhp.php (PHP reverse shell code)
Attacker location (IP lookup)	Shijiazhuang, China (approximate)
First SQL injection attempt	Packet 357 at 12:03:52.3 UTC: /search.php?search=book and 1=1; -- -
Hidden directory found	/admin/
VirusTotal result for the file	1 of 62 vendors flag it as malicious
Reverse shell connection	Not confirmed

5.2 Traffic overview

The capture has 88,862 packets. Most of them (83,418, or 93.9%) are HTTP, and there is no TLS, so all web traffic is readable. Almost all the traffic is between the attacker 111.224.250.131 and the server 73.124.22.98 (88,484 packets, 29 MB). A second outside IP, 170.40.150.126, exchanged only 256 packets with the server. The last 122 packets are UDP broadcast noise from 73.124.22.1.

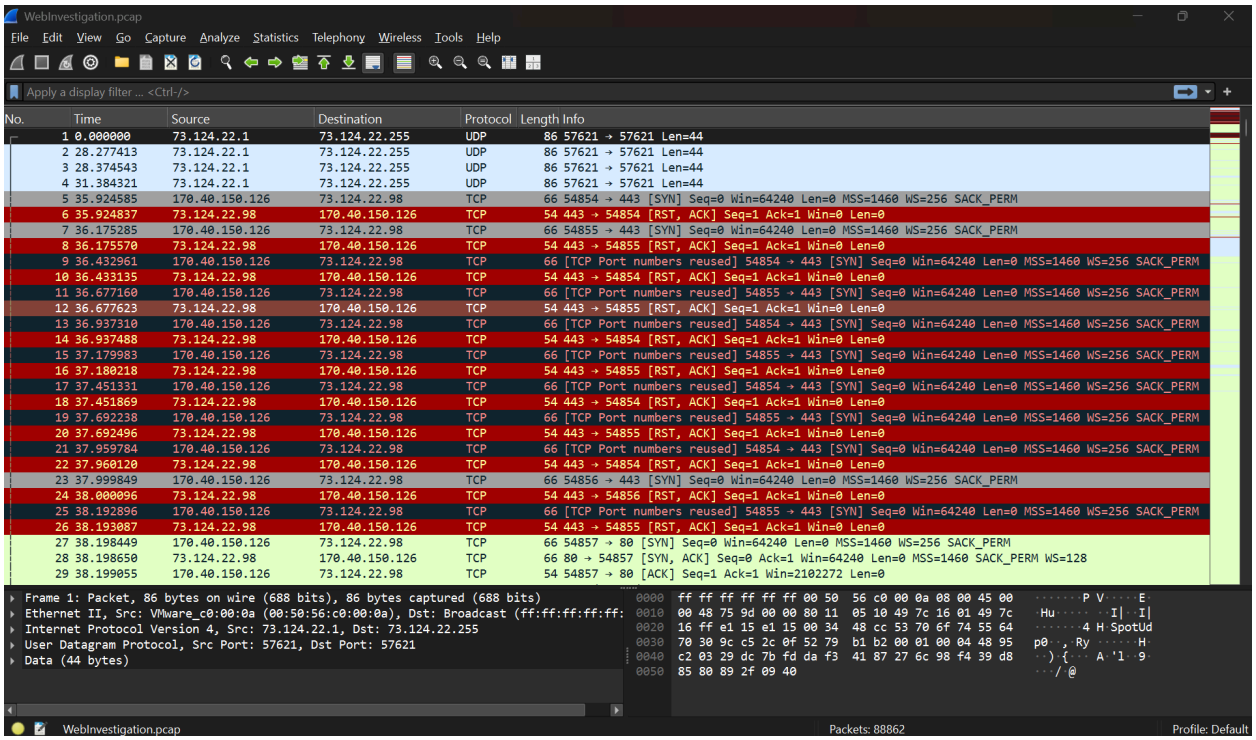


Figure 1: Packet list of WebInvestigation.pcap (88,862 packets)

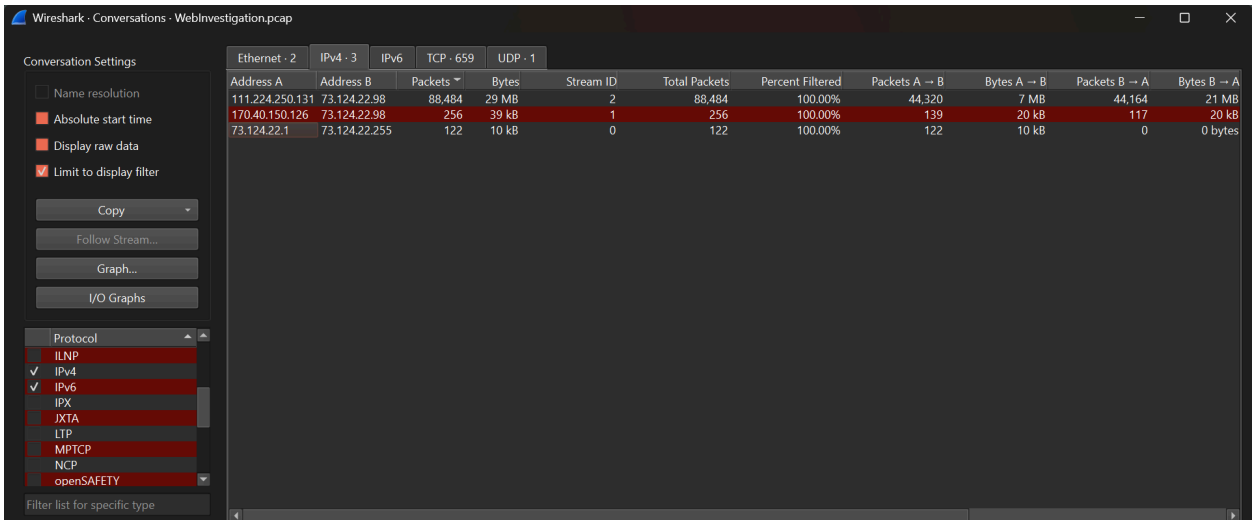


Figure 2: IPv4 conversations: the attacker and the server carry almost all traffic

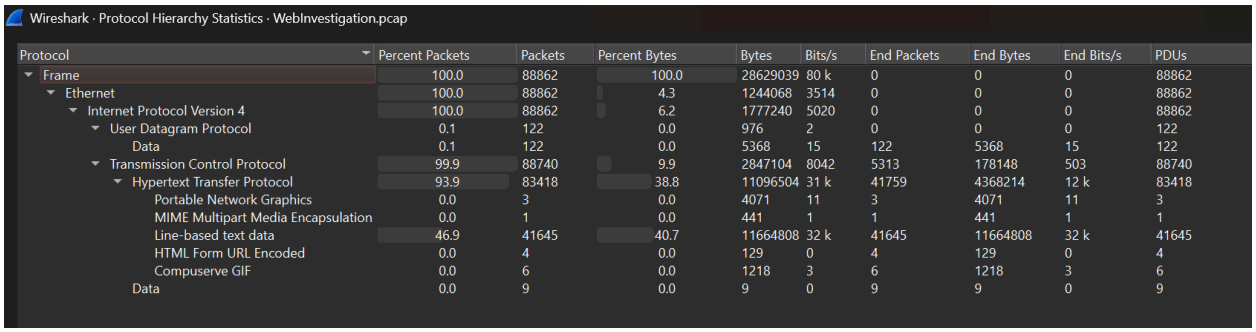


Figure 3: Protocol Hierarchy: HTTP is 93.9% of packets

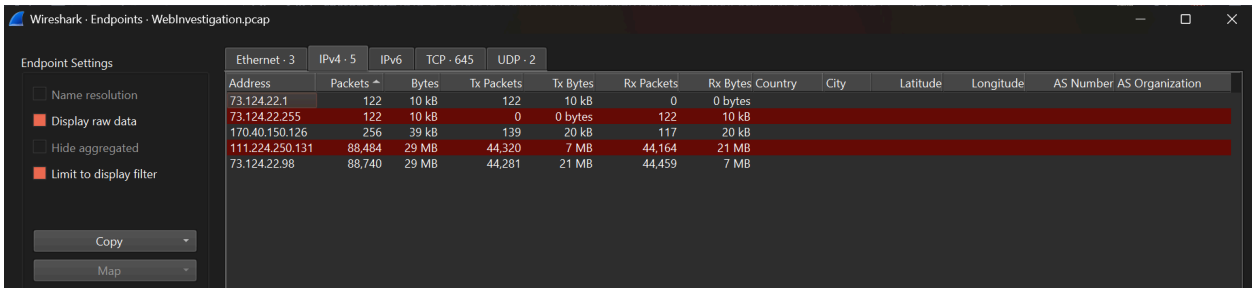


Figure 4: IPv4 endpoints

5.3 Normal traffic and scanning

170.40.150.126 is a normal visitor. It opens pages and searches for books like harry potter, Dracula and the hobbit, using a Windows browser. Its first attempts to connect on port 443 were reset because nothing on the server accepted connections on that port. We found nothing harmful from this IP.

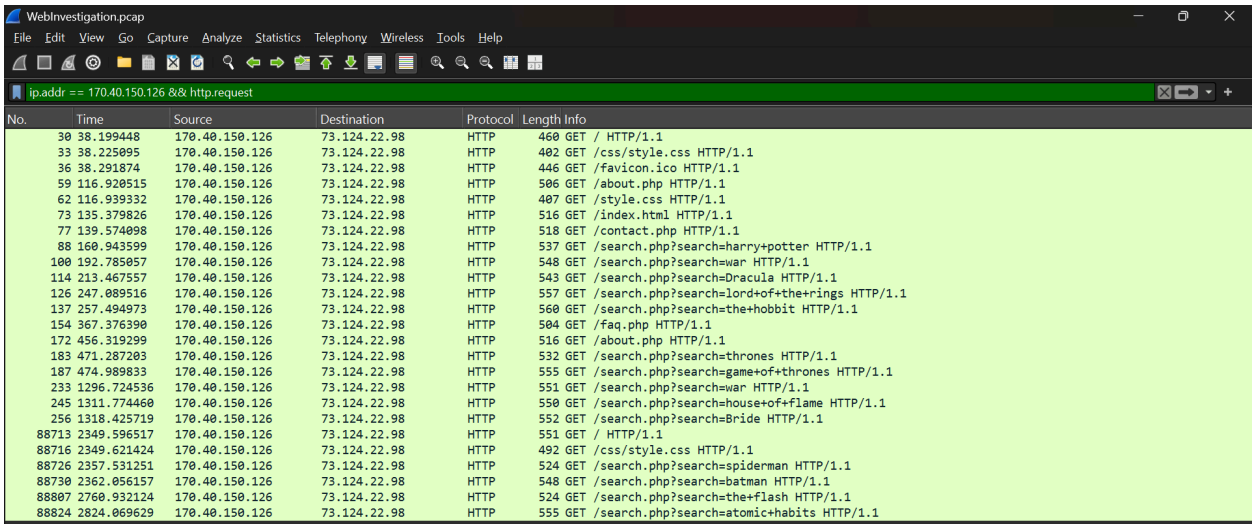


Figure 5: Normal browsing and book searches from 170.40.150.126

The attacker IP behaves very differently. It requests many pages that a normal user would never ask for, such as /phpinfo.php, /cachemgr.cgi, /admin.php, /xmlrpc.php and /~www. When we grouped the requests by User-Agent with tshark, we saw 41,527 requests from gobuster/3.6, 121 from sqlmap/1.8.3 and 36 from a Firefox-like browser. The Gobuster scan starts at about 12:12:20 UTC. A User-Agent can be faked, so we also looked at the number of requests and the pages requested. Together they point to an automated scan for hidden pages. The scan reached /admin at about 12:12:21 UTC, and the first request for /admin/login.php comes at about 12:12:58 UTC. This is how the attacker found the hidden admin area.

Wireshark · HTTP / Requests · WebInvestigation.pcap

Request Type	Cou	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Start
HTTP Requests by HTTP Host	41709				0.0150	100%	10.5000	1983.206
bookworldstore.com	41709				0.0150	100.00%	10.5000	1983.206
/search.php?search...	8				0.0000	0.02%	0.0100	1399.505
/admin/login.php	5				0.0000	0.01%	0.0100	2016.519
/index.html	4				0.0000	0.01%	0.0200	1982.112
/about.php	4				0.0000	0.01%	0.0100	116.921
/	4				0.0000	0.01%	0.0100	38.199
/search.php?search...	3				0.0000	0.01%	0.0100	192.785
/favicon.ico	3				0.0000	0.01%	0.0100	38.292
/faq.php	3				0.0000	0.01%	0.0100	367.376
/css/style.css	3				0.0000	0.01%	0.0100	38.225
/contact.php	3				0.0000	0.01%	0.0100	139.574
/xmlrpc_server.php	2				0.0000	0.00%	0.0200	1984.768
/xmlrpc.php	2				0.0000	0.00%	0.0200	1984.765
/style.css	2				0.0000	0.00%	0.0100	116.939
/search.php?search...	2				0.0000	0.00%	0.0200	1731.030
/search.php?search...	2				0.0000	0.00%	0.0100	1515.613
/robots.txt	2				0.0000	0.00%	0.0200	1983.587
/phpinfo.php	2				0.0000	0.00%	0.0200	1983.087
/info.php	2				0.0000	0.00%	0.0200	1982.139
/index.php	2				0.0000	0.00%	0.0200	1982.111
/icons/unknown.gif	2				0.0000	0.00%	0.0100	2702.792
/icons/blank.gif	2				0.0000	0.00%	0.0100	2702.781
/icons/back.gif	2				0.0000	0.00%	0.0100	2702.788
/cachemgr.cgi	2				0.0000	0.00%	0.0200	1980.201
/admin/uploads/	2				0.0000	0.00%	0.0100	2702.752
/admin/index.php	2				0.0000	0.00%	0.0100	2294.310
/admin.php	2				0.0000	0.00%	0.0200	1979.224
/admin.cgi	2				0.0000	0.00%	0.0200	1979.224
/admin	2				0.0000	0.00%	0.0100	1979.221
/~www.txt	1				0.0000	0.00%	0.0100	1978.773
/~www.php	1				0.0000	0.00%	0.0100	1978.773
/~www.js	1				0.0000	0.00%	0.0100	1978.769
/~www.html	1				0.0000	0.00%	0.0100	1978.773
/~www.cgi	1				0.0000	0.00%	0.0100	1978.769
/~www.bak	1				0.0000	0.00%	0.0100	1978.773
/~www.axd	1				0.0000	0.00%	0.0100	1978.769
/~www.asp	1				0.0000	0.00%	0.0100	1978.769
/~www	1				0.0000	0.00%	0.0100	1978.769

Display filter: Enter a display filter ...

Figure 6: HTTP request list showing scanner style requests

```
PS C:\Users\shkma> $pcap = "$HOME\Downloads\WebInvestigation\temp_extract_dir\WebInvestigation.pcap"
PS C:\Users\shkma> & "C:\Program Files\Wireshark\tshark.exe" -r $pcap -Y "http.request" -T fields -e ip.src -e http.user_agent | Grou
p-Object | Sort-Object Count -Descending

Count Name                     Group
----- ----                     -----
41527 111.224.250.131   gobust... {111.224.250.131      gobuster/3.6, 111.224.250.131   gobuster/3.6, 111.224.250.131  gobuste...
  121 111.224.250.131   sqlmap... {111.224.250.131      sqlmap/1.8.3#stable (https://sqlmap.org), 111.224.250.131      sqlmap/1....
   36 111.224.250.131   Mozill... {111.224.250.131      Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/11...
   25 170.40.150.126    Mozilla... {170.40.150.126      Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,...
```

Figure 7: Requests grouped by source IP and User-Agent (tshark)

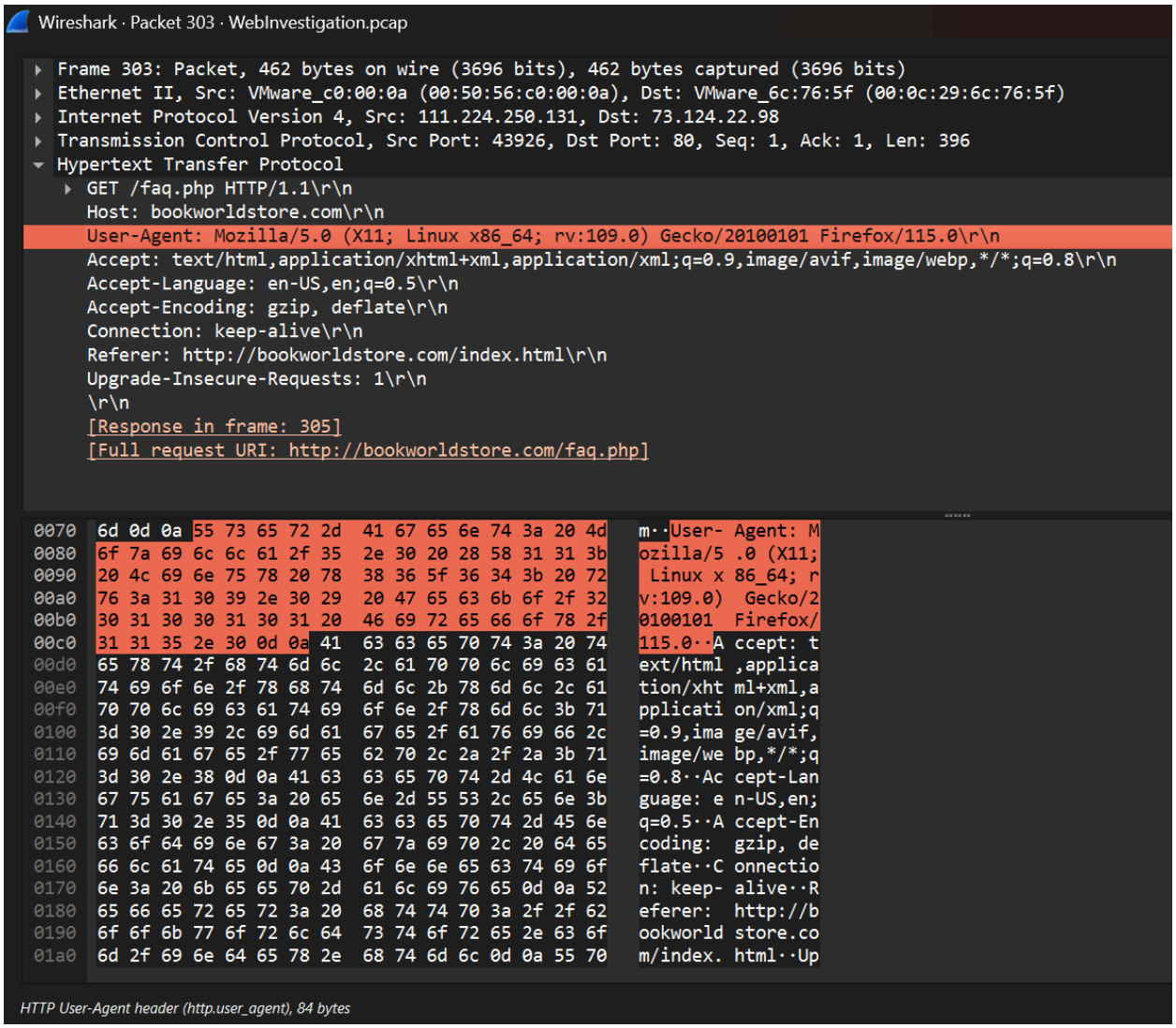


Figure 8: A browser-style request from the attacker IP (packet 303)

5.4 SQL injection and data exposure

The attacker sent many GET requests to /search.php with SQL code in the search value. The requests use UNION ALL SELECT and information_schema, and the User-Agent is sqlmap/1.8.3. The filter ip.src == 111.224.250.131 && http.request.uri contains “UNION” shows them.

The first SQL injection attempt is packet 357 at 12:03:52.3 UTC. Decoded, it is /search.php?search=book and 1=1; -- -. This is a simple test to see if the page accepts SQL code. The UNION requests shown below start later, at 12:07:34.2 UTC.

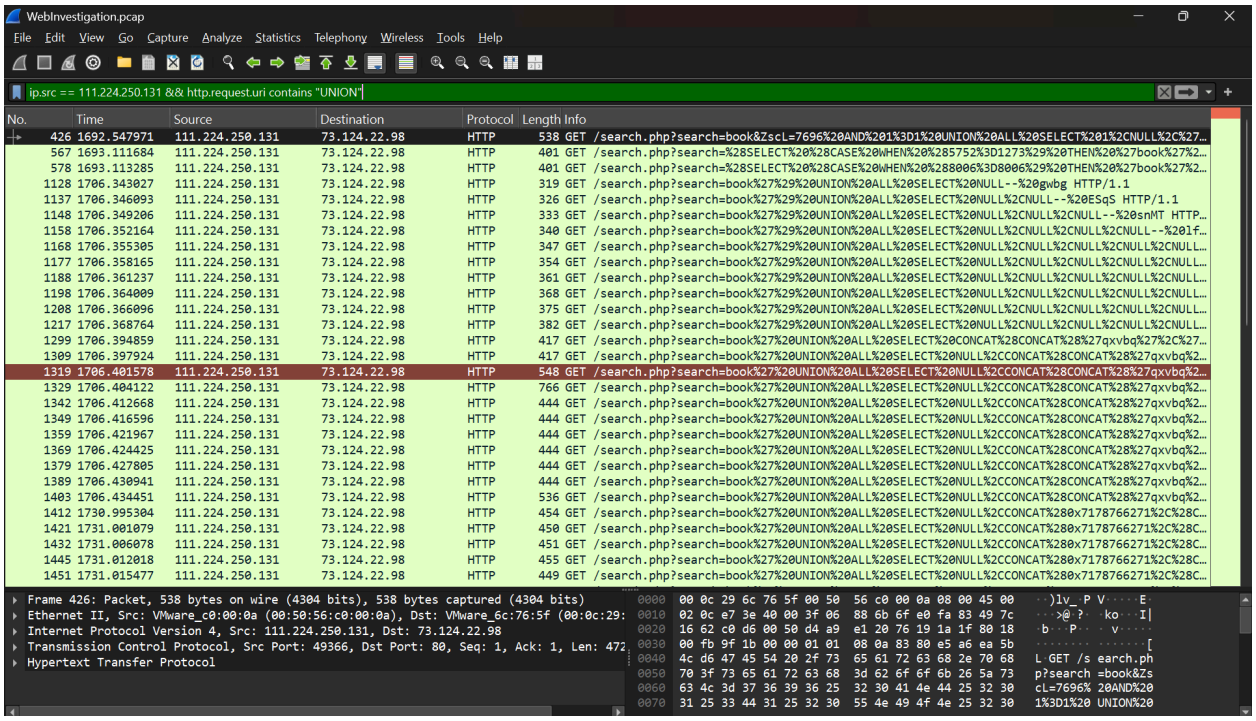


Figure 9: UNION based SQL injection requests from the attacker

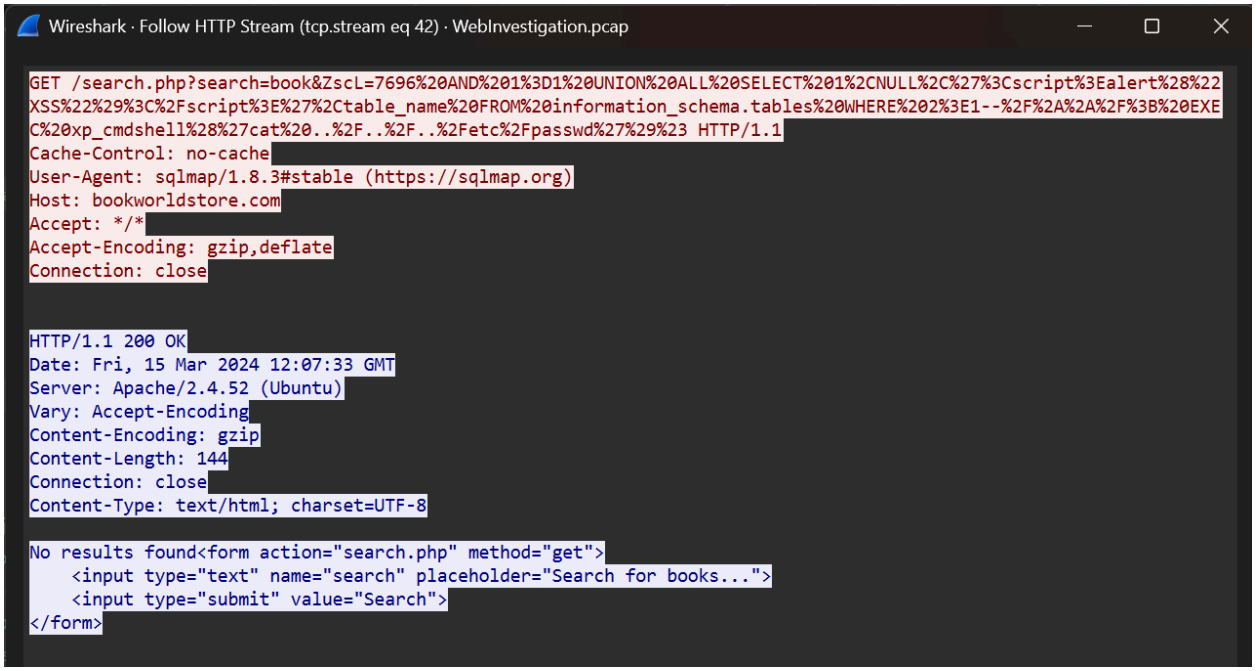


Figure 10: Followed HTTP stream showing the sqlmap User-Agent and payload

Many responses only say “No results found”. That shows the attempt, but not that it worked. Other queries listed the databases (INFORMATION_SCHEMA.SCHEMATA) and then the tables in bookworld_db. The table with the website users’ data is customers. One server response (packet 1624 at 12:09:40 UTC) contains 10 customer records from bookworld_db.customers:

names, emails, addresses, phone numbers and IDs. This confirms the attack worked and customer data was exposed.

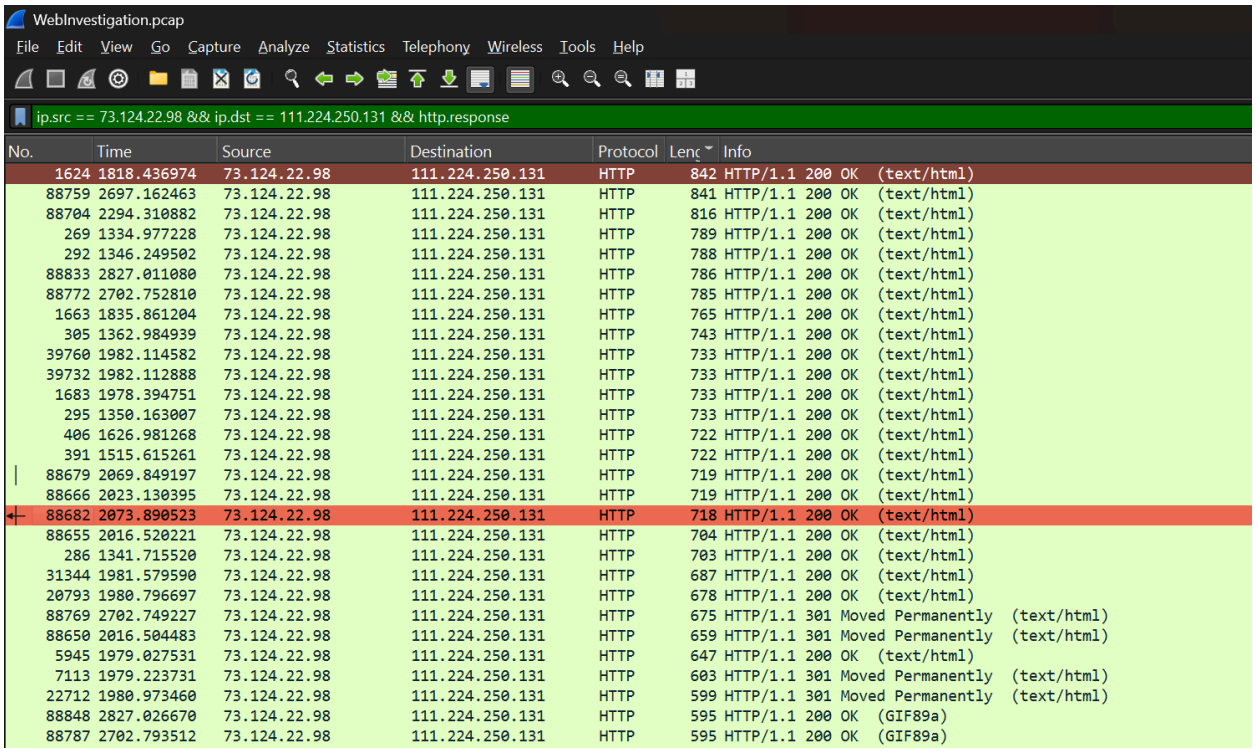


Figure 11: Server responses sent to the attacker, sorted by length

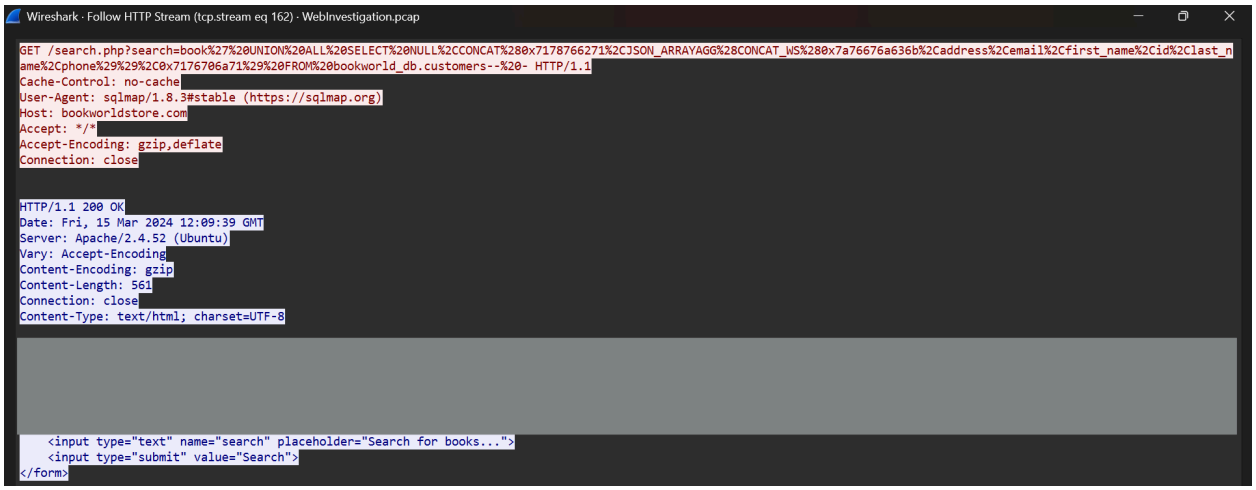


Figure 12: Response containing customer records (data redacted)

5.5 Admin login

The attacker then sent five POST requests. The first three login attempts to /admin/login.php failed. The server replied 200 OK with “Invalid username or password”. The fourth attempt got 302 Found, which sends the browser to index.php, and the next request returned the Admin Dashboard (200 OK). So the admin login worked. The dashboard has a file upload form. The

attacker used the admin account with a simple password, which we do not write here. The capture does not show how the attacker got the login details.

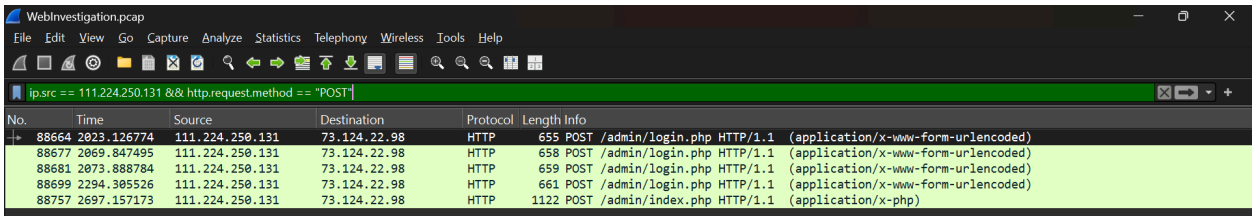


Figure 13: POST requests from the attacker (five in total)

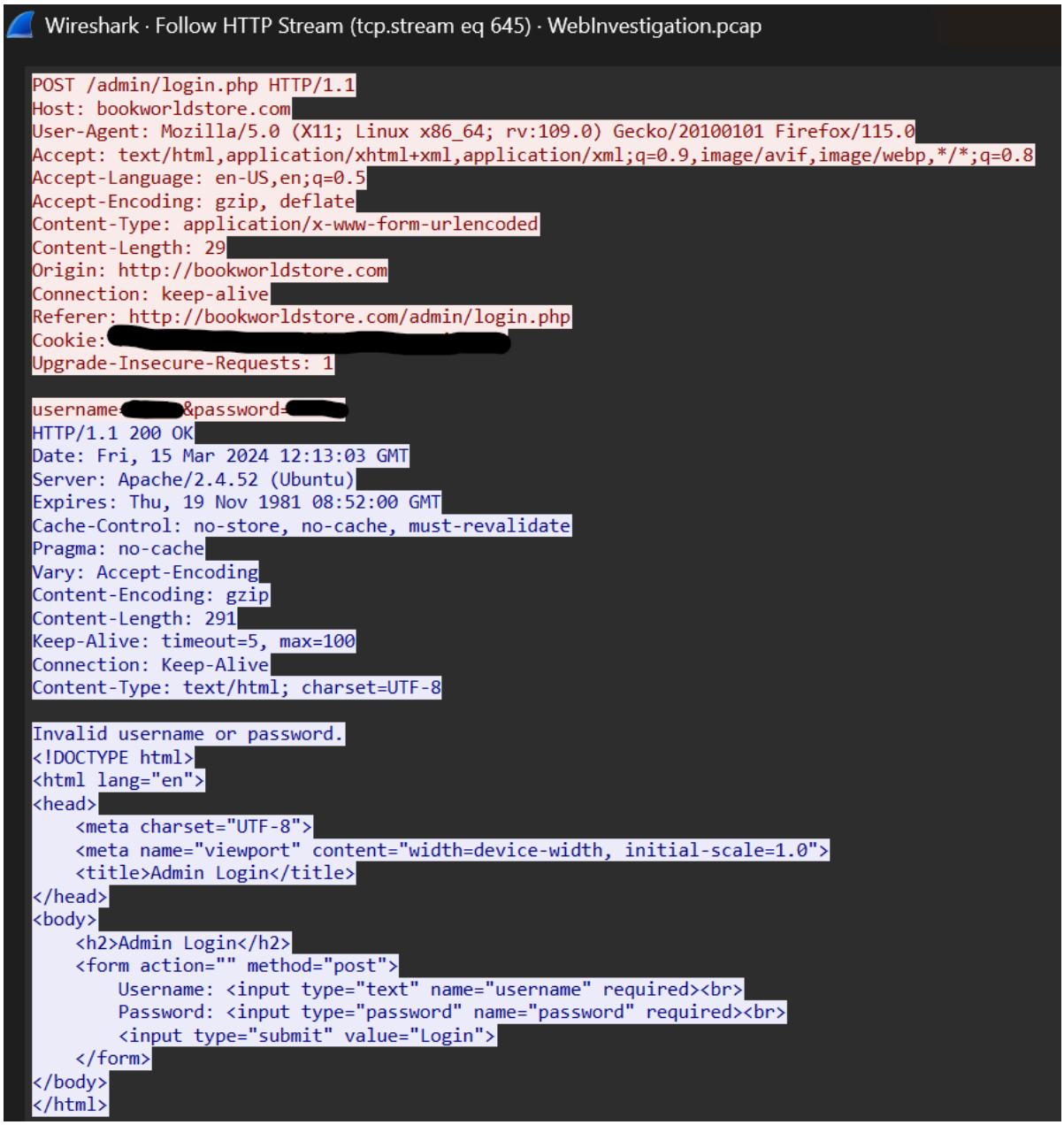


Figure 14: Failed login: "Invalid username or password"

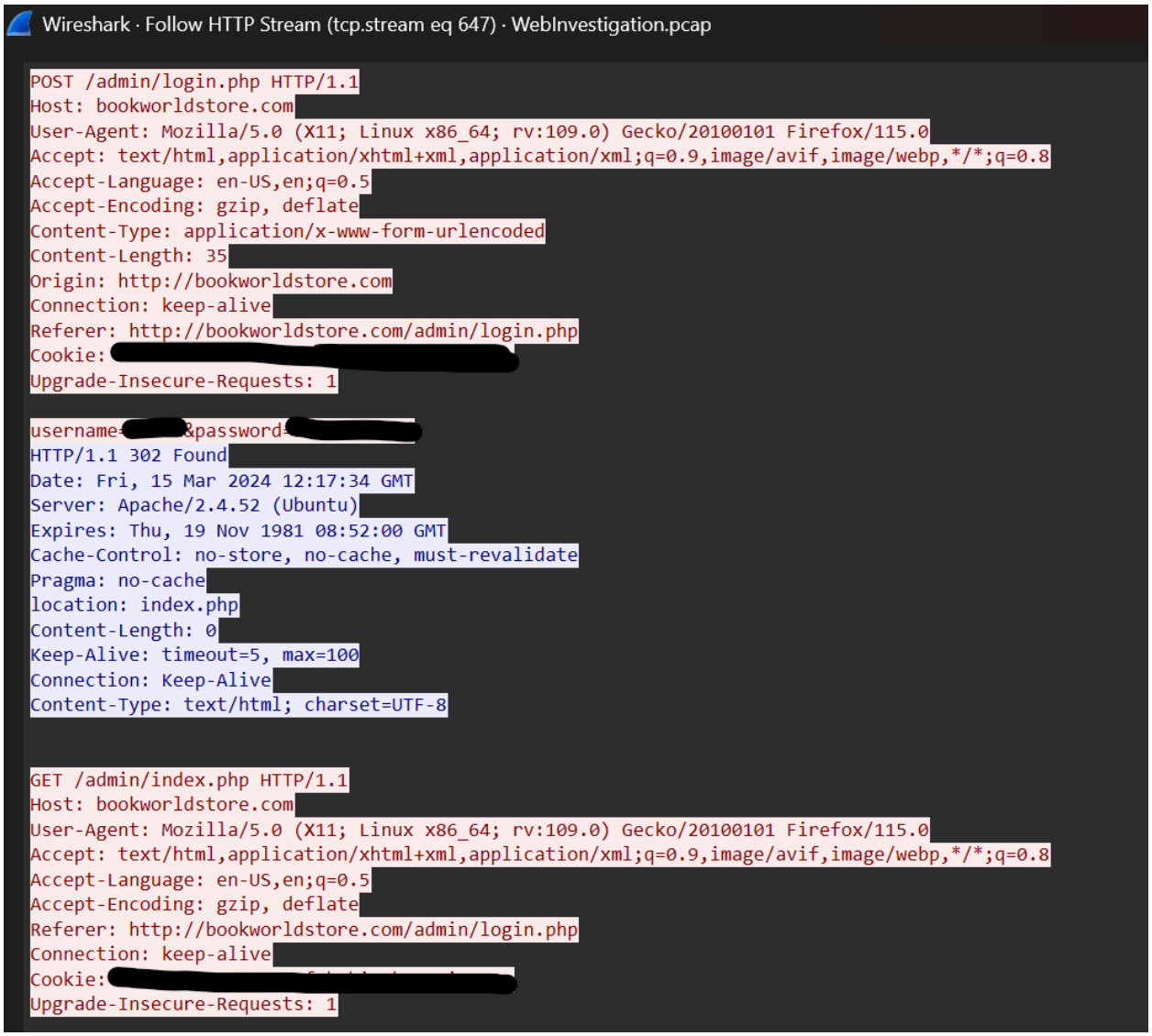


Figure 15: Successful login: 302 redirect to index.php, then the dashboard request

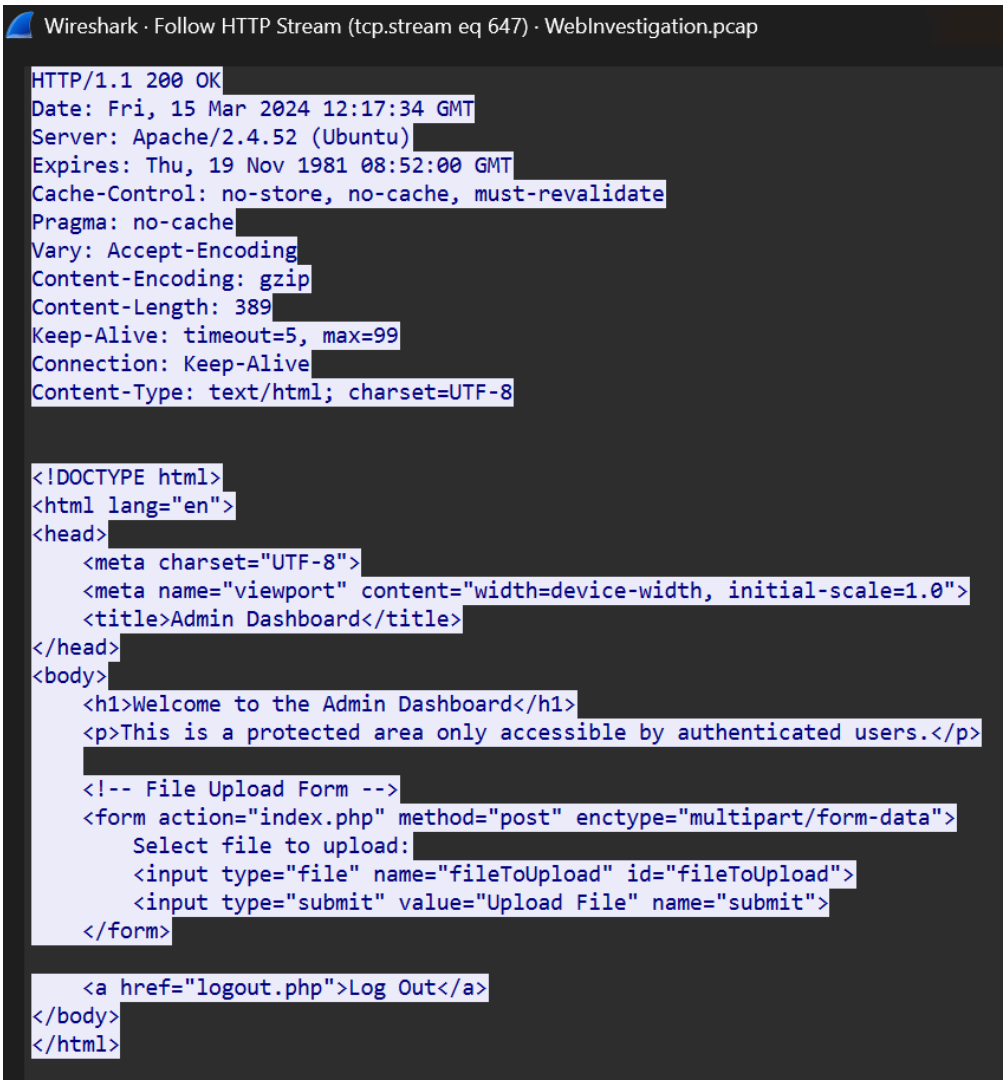


Figure 16: Admin Dashboard with the file upload form

5.6 Web shell upload

The fifth POST went to /admin/index.php as multipart/form-data. The uploaded file is NVri2vhp.php and its type is application/x-php. It holds one line of PHP that runs Bash and tries to connect back to 111.224.250.131 on port 443. The server replied 200 OK with “The file NVri2vhp.php has been uploaded”. So the server accepted a malicious PHP file.



Figure 17: File upload request with the reverse shell code and the server response

5.7 Use of the uploaded file

At 12:24:28.7 UTC the attacker requested /admin/uploads/NVri2vhp.php (packet 88790). The server replied HTTP 500 Internal Server Error (packet 88791). We then looked for the connection back with the filter ip.addr == 111.224.250.131 && tcp.port == 443 and found no packets. So the file was requested, but we cannot say the shell ran or connected.

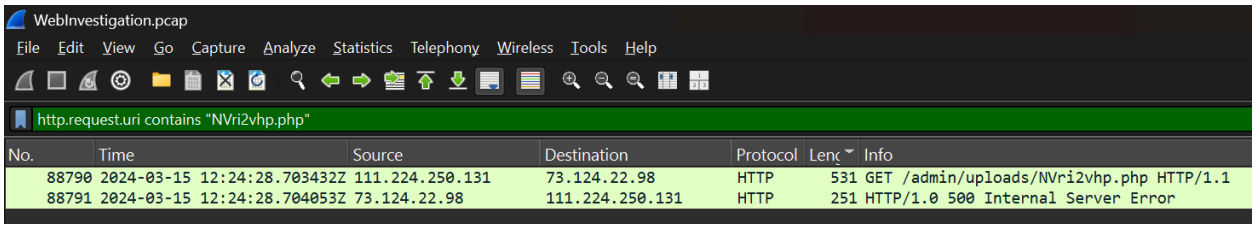


Figure 18: Request to the uploaded file and the HTTP 500 response

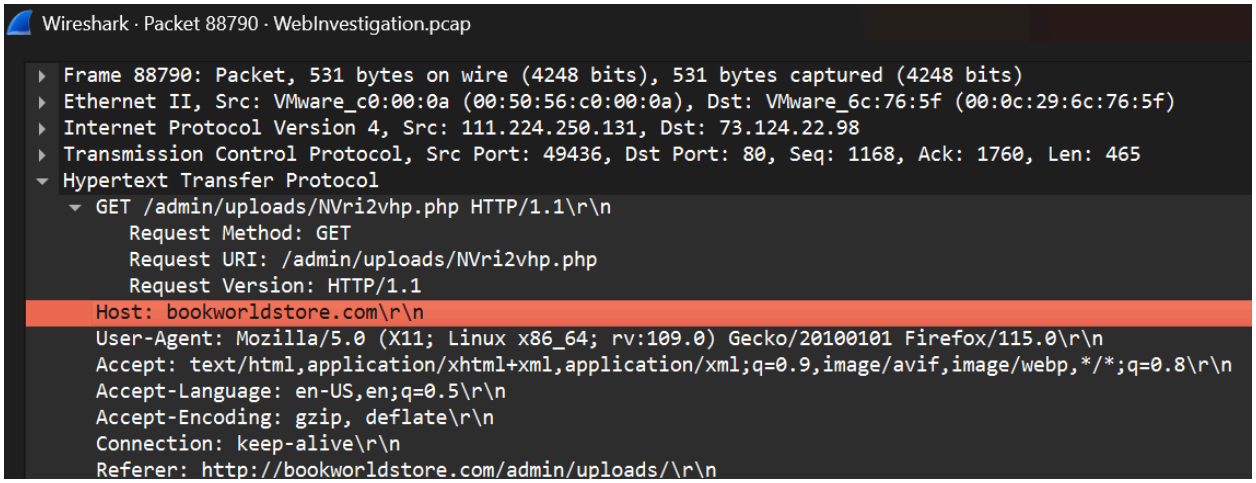


Figure 19: Packet 88790: GET /admin/uploads/NVri2vhp.php

5.8 File analysis

We opened the PCAP in NetworkMiner 3.2, which lists the files it can rebuild from the traffic (41,655 in total). Searching for NVri2vhp.php gives one result: frame 88757, 79 bytes, from 111.224.250.131 to 73.124.22.98 on port 80, labelled HttpPostMimeFileData. This matches the upload request we found in Wireshark. We only extracted the file. We never opened or ran it.

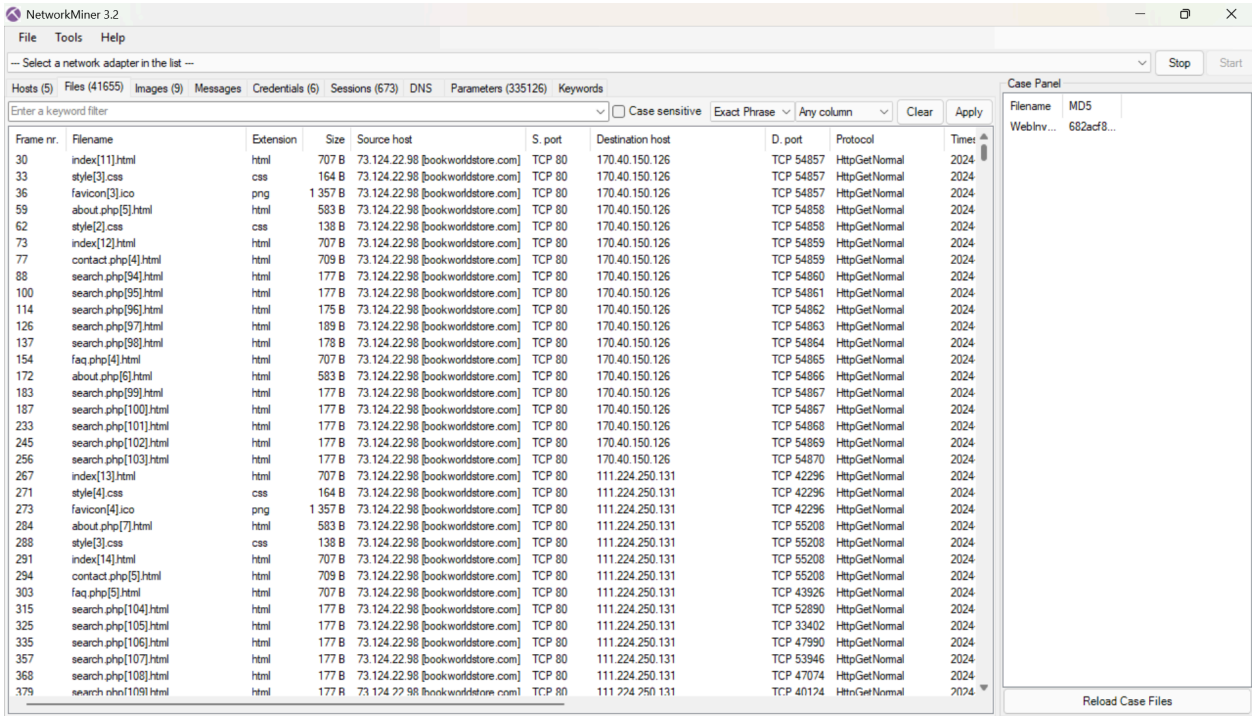


Figure 20: NetworkMiner Files tab for WebInvestigation.pcap

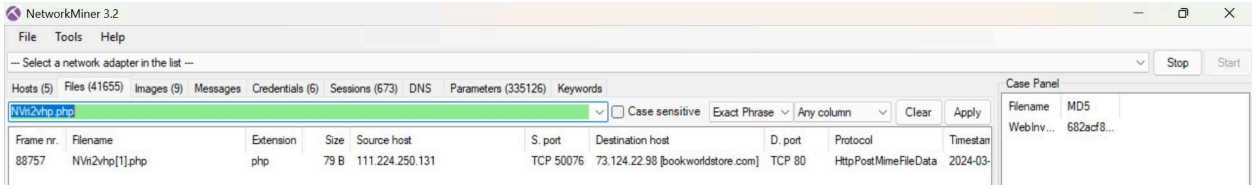


Figure 21: NetworkMiner search result: NVri2vhp.php in frame 88757

We then got the SHA256 hash of the file with PowerShell: 24D6D938AA8D87FC32577D3513FAD6FE2BF574EBC3D7967A429E52171D92A745.

We searched the hash on VirusTotal without uploading the file. One of 62 vendors flagged it as malicious (Kaspersky: HEUR:Backdoor.Shell.Revshell.a). VirusTotal also says the script runs a Bash command that makes a reverse shell to 111.224.250.131 on port 443. Only one vendor flagged it, so this supports our finding but does not prove it.

```
PS C:\Users\shkma\AppData\Local\NetworkMiner\AssembledFiles\111.224.250.131\TCP-50076> Get-FileHash -LiteralPath ".\NVri2vhp[1].php"
-Algorithm SHA256

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          24D6D938AA8D87FC32577D3513FAD6FE2BF574EBC3D7967A429E52171D92A745       C:\Users\shkma\AppData\Local\...
```

Figure 22: SHA256 hash of the extracted file (PowerShell Get-FileHash)

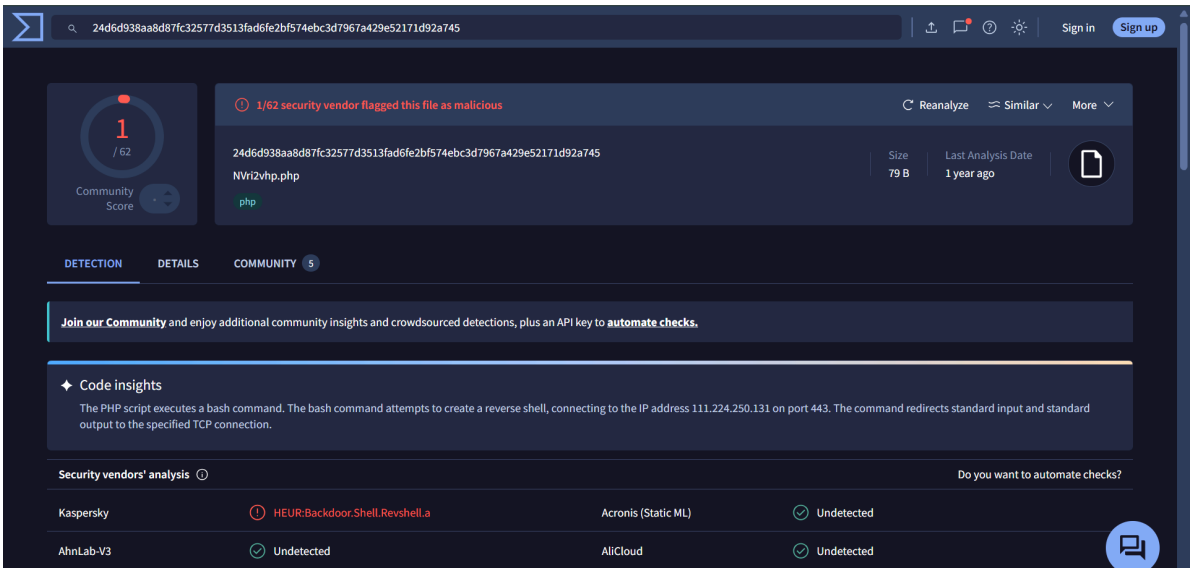


Figure 23: VirusTotal result for the hash

5.9 Traffic spikes

The I/O graph shows short bursts of attacker traffic that match the sqlmap activity. A burst alone does not prove an attack, so we used the graph only as background next to the HTTP evidence.

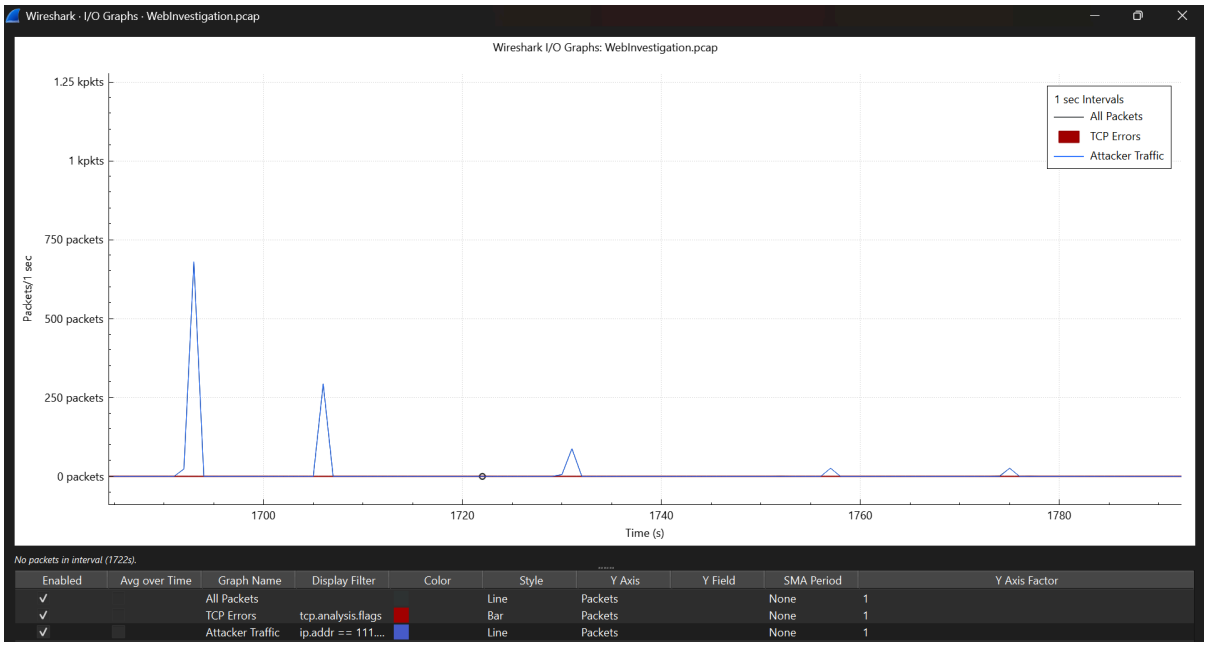


Figure 24: I/O graph: all packets, TCP errors and attacker traffic

6. Timeline

All times are UTC on 15 March 2024, taken from the Wireshark packet timestamps.

Time (UTC)	Event	Evidence
12:03:52.3	First SQL injection attempt on /search.php	Packet 357
12:07:34.2	sqlmap UNION ALL SELECT requests seen	Packet 426
12:09:40.1	Server returns customer records	Packet 1624
About 12:12:20	Gobuster directory scan starts	HTTP request statistics
About 12:12:58	Attacker requests /admin/login.php (admin area found)	HTTP request statistics
12:13:04.8	Failed admin login 1	Packet 88664
12:13:51.5	Failed admin login 2	Packet 88677
12:13:55.6	Failed admin login 3	Packet 88681
12:17:36.0	Successful admin login, dashboard loads	Packets 88699, 88704
12:24:18.8	NVri2vhp.php uploaded	Packet 88757
12:24:28.7	Uploaded file requested, server returns 500	Packets 88790, 88791

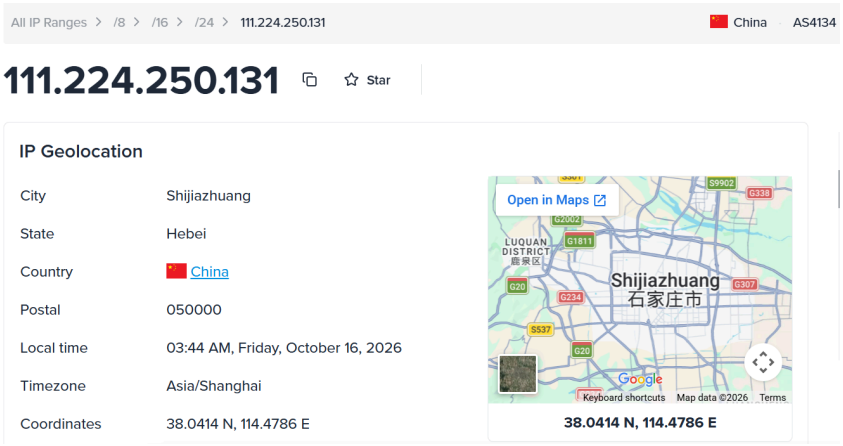
7. Indicators of Compromise

Our script extract_iocs.py reads the exported tshark output and writes IP addresses, URLs and domains to iocs.csv (it also picks up hashes if they are present in the text). The table below lists the key indicators. The full CSV is in the repository.

Type	Value	Notes
Attacker IP	111.224.250.131	Source of attack traffic. IPinfo reports this IP's geolocation as Shijiazhuang, China. IP geolocation is approximate and does not establish the attacker's actual physical location, as VPNs, proxies and other routing arrangements can affect the result.
Victim IP	73.124.22.98	BookWorld web server
Domain	bookworldstore.com	Host header of all requests
User-Agent	gobuster/3.6	Directory scanning
User-Agent	sqlmap/1.8.3#stable	SQL injection
File	NVri2vhp.php	Uploaded PHP reverse shell
Path	/admin/uploads/NVri2vhp.php	Where the file was requested
Port	443/tcp	Reverse shell target (no traffic seen)

SHA256	24D6D938AA8D87FC32577D3513FAD6FE2BF574EBC3D7967A429E52171D92A745	NVri2vhp.php as extracted by NetworkMiner (79 bytes). VirusTotal: 1 of 62 vendors flag it (Kaspersky: HEUR:Backdoor.Shell.Revshell.a).

We checked the script output in PowerShell. The CSV holds 1 domain, 3 IP addresses and 152 URLs. The three IPs are 111.224.250.131 (attacker), 73.124.22.98 (server) and 170.40.150.126 (normal visitor). The visitor is in the CSV because the script lists every IP it sees, but it is not an indicator of compromise. The hash is not in the CSV because we took it from NetworkMiner, not from the tshark export.




```
PS C:\Users\shkma\Desktop\college\SEM V\CFL\web-investigation-dfir> Import-Csv .\investigations\web-investigation\iocs.csv |
>>     Group-Object type |
>>     Select-Object Name, Count

Name   Count
----   -----
domain     1
ip         3
url      152
```

Figure 25 and 26: Attacker's IP address in IPinfo and Count of IOC types in iocs.csv

8. MITRE ATT&CK Mapping

Phase	Technique	What we saw
Reconnaissance	Wordlist Scanning (T1595.003)	Gobuster scan with 41,527 requests
Initial Access	Exploit Public-Facing Application (T1190)	SQL injection on /search.php
Initial Access, Persistence	Valid Accounts (T1078)	Admin login worked
Persistence	Web Shell (T1505.003)	NVri2vhp.php uploaded
Execution (attempted)	Unix Shell (T1059.004)	Bash reverse shell code, execution not confirmed

9. Impact Assessment

- Confidentiality: customer personal data was exposed through SQL injection. At least 10 records are visible in one response and other responses may hold more.
- Access: the attacker gained administrator access to the site.
- Integrity: a malicious PHP file was placed on the server.
- Control of the server: not confirmed, because no reverse shell connection was seen.

10. Recommendations

What the evidence showed	Recommended fix
The search page accepted SQL code	Use parameterised queries and validate all input on /search.php.
41,527 scanner requests and sqlmap traffic went unchecked	Put a web application firewall and rate limiting in front of the site and alert on scanner User-Agents such as gobuster and sqlmap.
The admin area was reachable from anywhere and used a simple password	Use strong admin passwords and MFA, lock accounts after failed logins and limit /admin to trusted IP addresses.
The upload form accepted a PHP file	Allow only safe file types, check file content, store uploads outside the web root and block .php files.
Uploaded scripts could be requested from /admin/uploads/	Turn off script execution in the upload folder.
Nothing raised an alert before the damage	Keep web, database and host logs and alert on unusual database queries and on outbound connections from the web server.
Customer data was exposed	Tell affected customers, check the database for other exposed data and reset credentials.

11. Conclusion

The capture shows a clear attack path: scanning, SQL injection that exposed customer data, a successful admin login and the upload of a PHP reverse shell. The packet evidence confirms each of these steps except the last one, because the capture does not show whether the reverse shell connected. NetworkMiner and VirusTotal supported the file analysis and every finding is tied to a packet or a screenshot. This project gave us practice in network forensics with Wireshark, tshark and NetworkMiner, in building a timeline from packets and in writing findings.

12. References

- CyberDefenders, Web Investigation Lab (Network Forensics): https://cyberdefenders.org/blueteam-ctf-challenges/web-investigation/
- MITRE ATT&CK, attack.mitre.org

- Wireshark User Guide, wireshark.org
- NetworkMiner, netresec.com/?page=NetworkMiner
- VirusTotal, virustotal.com & IPinfo, ipinfo.io

Appendix A: Lab Questions and Answers

These are the nine questions of the CyberDefenders lab, with our answers and where the evidence is in this report.

Lab question	Answer	Where
Q1. Attacker IP	111.224.250.131	Section 5.2
Q2. Attacker city	Shijiazhuang, China (IPinfo, approximate)	Section 7
Q3. Vulnerable script	search.php	Section 5.4
Q4. First SQL injection request	/search.php?search=book and 1=1; -- - (packet 357)	Section 5.4
Q5. Request that read the databases	A UNION ALL SELECT on INFORMATION_SCHEMA.SCHEMATA sent to /search.php	Section 5.4
Q6. Table with the user data	customers (in bookworld_db)	Section 5.4
Q7. Hidden directory found	/admin/	Sections 5.3 and 5.5
Q8. Credentials used	Username admin, with a simple password (not reproduced here)	Section 5.5
Q9. Malicious script uploaded	NVri2vhp.php	Sections 5.6 to 5.8

